

Memoria Técnica: Ingeniería de Datos y Modelación Hidrológica en App-SCALL

Sistema Estocástico de Dimensionamiento de Captación de
Aguas Lluvias

Autor: Diego Elsholz

Proyecto de Ingeniería Hidrológica
Fundación Amulén — La Fundación del Agua

Versión 2.1
23 de mayo de 2026

Índice

1. Resumen Ejecutivo	2
2. Fuente de Información: La Base de Datos (CR)²	2
2.1. Origen de las mediciones	2
2.2. Alcance espacial y temporal	3
3. Ingeniería y Tratamiento de Datos	3
3.1. Carga por chunks	3
3.2. Saneamiento de coordenadas	3
3.3. Gestión de datos faltantes	3
3.4. Control de calidad por estación	4
3.5. Caché de datos	4
4. Inteligencia Geoespacial	4
4.1. Selección de estación meteorológica	4
4.2. Corrección altitudinal de la distancia	4
4.3. Ingreso de ubicación	5
5. Corrección Orográfica de la Precipitación	5
6. Algoritmo Hidrológico: Modelo YAS (<i>Yield After Spillage</i>)	6
6.1. 1. Captación bruta (Q_t)	6
6.2. 2. Volumen previo al rebalse (V_{temp})	6
6.3. 3. Cálculo de rebalse (S_t)	7
6.4. 4. Abastecimiento efectivo y déficit (Y_t)	7
6.5. 5. Almacenamiento final (V_t)	7
6.6. Demanda diaria	7
7. Análisis de Escenarios Críticos y Selección Estocástica	7
7.1. Año Seco (Percentil P5)	8
7.2. Año Lluvioso (Percentil P95)	8
7.3. Año Normal: Algoritmo de Doble Filtro Estadístico	8
8. Curva de Optimización: El “Sweet Spot” del Diseño	9
8.1. Vectorización del bucle YAS	9
8.2. Identificación del punto óptimo	9
9. Mapa de Factibilidad Nacional	10
9.1. Precipitación media anual por estación	10
9.2. Triangulación de Delaunay	10
9.3. Valor de cada triángulo	10
9.4. Escala de colores	11
9.5. Mapa de cobertura estimada	11
10. Interfaz de Usuario e Interoperabilidad	11
10.1. Pestañas de resultados	12

10.2. Exportación de datos	12
11. Informe PDF de Viabilidad	12
11.1. Identidad visual	12
11.2. Estructura del informe	13
12. Historial de Simulaciones y Persistencia de Datos	13
12.1. Arquitectura de persistencia	13
12.2. Esquema de la base de datos	14
12.3. Serialización de datos	14
12.4. Gestión multi-usuario	14
12.5. Funcionalidades del historial	15
13. Análisis de Seguridad y Autonomía Hídrica	15
14. Arquitectura del Software	16
14.1. Stack tecnológico	16
15. Despliegue	17
16. Conclusión	17
Referencias Bibliográficas	17

1 Resumen Ejecutivo

App-SCALL es una plataforma de software desarrollada en Python (Streamlit) para la simulación y dimensionamiento de sistemas de captación de aguas lluvias (SCALL). A diferencia de calculadoras simplificadas basadas en promedios, App-SCALL utiliza **registros climáticos históricos diarios reales** (2000–2020) y un algoritmo de balance de masas diario para entregar resultados con un nivel de confianza estadística superior al 95 %.

La aplicación está disponible en línea en:

<https://scallclaude-a6ziabnwkurju5e24brqke.streamlit.app>

Los principales módulos del sistema son:

- **Motor de simulación hidrológica:** balance diario bajo el modelo YAS (*Yield After Spillage*) con corrección orográfica de la precipitación.
- **Selección estocástica de escenarios:** extracción de años reales seco (P5), normal (mediana + SSE) y lluvioso (P95) desde 20 años de registro.
- **Optimización automática:** algoritmo Kneedle para identificar el “Sweet Spot” de capacidad del estanque.
- **Mapa de factibilidad nacional:** triangulación de Delaunay sobre ~ 400 estaciones CR² con precipitación media anual interpolada.
- **Generación de informes:** exportación a PDF (identidad visual Amulén, incluye gráfico de curva óptima) y Excel con balance diario completo.
- **Historial persistente multi-usuario:** almacenamiento automático de simulaciones en la nube (Supabase/PostgreSQL), con gestión por nombre de usuario, recarga de simulaciones previas y descarga masiva en formato ZIP.

2 Fuente de Información: La Base de Datos (CR)²

El pilar de confiabilidad de la aplicación es el uso de la base de datos del Centro de Ciencia del Clima y la Resiliencia (CR)² de la Universidad de Chile.

2.1 Origen de las mediciones

Los registros provienen de una extensa red física de estaciones meteorológicas operadas históricamente por la Dirección Meteorológica de Chile (DMC) y la Dirección General de Aguas (DGA), cuyos datos crudos fueron consolidados y estandarizados por el (CR)² en el producto `cr2_prDaily_2020`.

2.2 Alcance espacial y temporal

- **Estaciones:** más de 1.000 estaciones a lo largo de Chile continental e insular.
- **Período analizado: 2000–2020** (20 años). Esta decisión metodológica responde a dos criterios:
 1. La calidad y densidad de datos es sustancialmente mayor desde el año 2000.
 2. El período incluye la Mega-Sequía en Chile Central (2010–2019), por lo que el modelo es más conservador y representativo del clima actual y proyectado.
- **Resolución temporal:** precipitación diaria (mm/día por estación).
- **Corte en 2020:** las restricciones de movilidad impuestas por la pandemia COVID-19 generaron interrupciones masivas en la lectura manual de pluviómetros a nivel nacional a partir de 2020, afectando la calidad de los datos posteriores.

3 Ingeniería y Tratamiento de Datos

Debido a que los archivos climáticos crudos presentan múltiples inconsistencias, App-SCALL implementa una robusta capa de limpieza y validación (*data cleaning*):

3.1 Carga por chunks

Para manejar el archivo principal (`cr2_prDaily_2020.txt.gz`, ~5,3 MB comprimido) sin colapsar la memoria RAM disponible en Streamlit Cloud (1 GB), la app lo procesa en bloques de 10.000 filas mediante `pd.read_csv(..., chunksize=10000)`, filtrando dinámicamente solo el período 2000–2020.

3.2 Saneamiento de coordenadas

El código implementa la función `arreglar_coordenada()` que detecta y corrige errores de digitación en las latitudes y longitudes del archivo original: puntos decimales omitidos, formatos de cadena incorrectos y valores fuera de rango geográfico para Chile ($-17,5 \leq \phi \leq -56$, $-76 \leq \lambda \leq -66$).

3.3 Gestión de datos faltantes

Los valores nulos están etiquetados como -9999 en climatología. El sistema los identifica y los trata como `NaN` (valor ausente en Pandas). Adicionalmente, los registros negativos se “bloquean” mediante un *clip* a cero (`clip(lower=0)`) para evitar errores matemáticos en el acumulado del estanque.

3.4 Control de calidad por estación

El sistema permite al usuario definir un umbral de calidad (por defecto 80 %). La aplicación analiza mes a mes cada estación meteorológica y descarta automáticamente aquellas que no alcancen el porcentaje mínimo de meses con datos válidos dentro del período 2000–2020, evitando simulaciones basadas en información incompleta.

Formalmente, una estación e es aceptada si:

$$\frac{\text{meses válidos}_e}{\text{meses totales}} \geq \frac{\text{umbral}}{100} \quad (1)$$

3.5 Caché de datos

Todas las funciones de carga de datos están decoradas con `@st.cache_data(ttl=3600)`, lo que almacena los resultados en memoria durante una hora. Esto evita releer y reprocesar el archivo CR^2 en cada interacción del usuario, reduciendo el tiempo de respuesta de ~ 8 segundos a $< 0,1$ segundos en consultas subsecuentes.

4 Inteligencia Geoespacial

4.1 Selección de estación meteorológica

Para determinar la pluviometría del lugar del proyecto, App-SCALL utiliza la **distancia de Haversine** implementada de forma vectorizada en NumPy. Esto permite calcular, en milisegundos, la distancia entre las coordenadas indicadas por el usuario y todas las estaciones de Chile simultáneamente.

La fórmula de Haversine para la distancia entre dos puntos en la esfera terrestre es:

$$d = 2R \cdot \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \phi}{2} \right) + \cos \phi_1 \cos \phi_2 \sin^2 \left(\frac{\Delta \lambda}{2} \right)} \right) \quad (2)$$

donde $R = 6,371$ km es el radio terrestre medio, ϕ la latitud y λ la longitud en radianes.

4.2 Corrección altitudinal de la distancia

La app no selecciona simplemente la estación más cercana en distancia horizontal. Aplica una **corrección altitudinal** que penaliza las estaciones con gran diferencia de altura respecto al proyecto, ya que la lluvia varía significativamente con la altitud en terrenos montañosos:

$$d_{\text{efectiva}} = d_{\text{Haversine}} + k \cdot \frac{|\Delta h|}{1000} \quad (3)$$

donde Δh es la diferencia de altitud en metros entre la estación y el proyecto, y k es el coeficiente de penalización altitudinal (por defecto $k = 5$ km por cada 100 m de desnivel).

4.3 Ingreso de ubicación

El usuario puede ingresar las coordenadas del proyecto de dos formas:

1. **Coordenadas manuales:** ingreso directo de latitud y longitud decimal.
2. **Link de Google Maps:** la app extrae automáticamente las coordenadas del URL mediante expresiones regulares, soportando los formatos `@lat,lon`, `?q=lat,lon` y `ll=lat,lon`.

La altitud se puede obtener automáticamente consultando la API pública OpenTopo-Data (SRTM 90m), con respaldo en Open-Meteo Elevation API.

5 Corrección Orográfica de la Precipitación

Una limitación fundamental de cualquier sistema basado en estaciones es que la precipitación medida en la estación no necesariamente representa la precipitación en el sitio del proyecto, especialmente cuando hay diferencias de altitud significativas.

App-SCALL implementa un **gradiente orográfico** configurable por el usuario. El parámetro define el porcentaje de variación de lluvia por cada 100 m de desnivel entre la estación y el proyecto:

$$P_{\text{proyecto}} = P_{\text{estación}} \times \left(1 + \frac{\gamma}{100} \cdot \frac{\Delta h}{100} \right) \quad (4)$$

donde:

- $P_{\text{estación}}$ = precipitación diaria registrada en la estación (mm/día)
- γ = gradiente orográfico (%/100 m de desnivel), configurable entre -15% y $+15\%$
- $\Delta h = h_{\text{proyecto}} - h_{\text{estación}}$ (m), positivo si el proyecto está más alto

El multiplicador final aplicado a toda la serie diaria es:

$$M = 1 + \frac{\gamma \cdot \Delta h}{10000} \quad (5)$$

Referencia rápida de uso:

Situación topográfica	Gradiente sugerido
Valle o costa plana	0 %
Precordillera (proyecto más alto)	+3 % a +7 %
Cordillera alta (proyecto más alto)	+5 % a +12 %
Sotavento / sombra de lluvia	−5 % a −10 %

Nota: los valores anteriores son orientativos. No existe actualmente una fuente oficial (CR² o DGA) que calibre gradientes orográficos a escala local para Chile. Se recomienda usar el valor por defecto (0 %) salvo que se disponga de datos locales que justifiquen el ajuste.

6 Algoritmo Hidrológico: Modelo YAS (*Yield After Spillage*)

La modelación estocástica del estanque se rige por la lógica matemática **YAS** (*Yield After Spillage*), el algoritmo de balance de masas diario recomendado por la literatura especializada [1] por ser el método más conservador y preciso para evitar la sobreestimación del recurso hídrico disponible.

Bajo el algoritmo YAS, durante el intervalo temporal de un día, el volumen de lluvia captado ingresa al estanque y el posible rebalse ocurre **antes** de que el usuario extraiga la demanda de agua. El estado del estanque con capacidad máxima C en el día t se define mediante el siguiente sistema secuencial de ecuaciones:

6.1 1. Captación bruta (Q_t)

Se calcula el volumen recolectado en el día t a partir de la precipitación corregida:

$$Q_t = P_t \times M \times A \times C_e \quad (6)$$

donde P_t es la precipitación diaria de la estación ($\text{mm} \equiv \text{L}/\text{m}^2$), M es el multiplicador orográfico (ver Sección 5), A es el área de la techumbre (m^2) y C_e es el coeficiente de eficiencia de captación (adimensional, $0,5 \leq C_e \leq 1,0$).

6.2 2. Volumen previo al rebalse (V_{temp})

Se calcula el almacenamiento temporal sumando el agua del día anterior más la captación actual:

$$V_{\text{temp}} = V_{t-1} + Q_t \quad (7)$$

6.3 3. Cálculo de rebalse (S_t)

Si el volumen temporal excede la capacidad nominal del estanque (C), el exceso se pierde:

$$S_t = \text{máx}(0, V_{\text{temp}} - C) \quad (8)$$

6.4 4. Abastecimiento efectivo y déficit (Y_t)

La extracción para cubrir la demanda diaria (D_t) se contabiliza solo sobre el agua que logró retenerse en el estanque tras el rebalse:

$$Y_t = \text{mín}(D_t, V_{\text{temp}} - S_t) \quad (9)$$

Si la demanda excede el volumen disponible, se genera un déficit. **Este déficit diario no es acumulativo**: el modelo asume un abastecimiento complementario externo (red pública, camión aljibe) que suple exactamente el volumen faltante de ese día, sin arrastrar deuda hídrica al día siguiente.

6.5 5. Almacenamiento final (V_t)

El volumen real que se transfiere al día siguiente es:

$$V_t = V_{\text{temp}} - S_t - Y_t = \text{máx}(0, \text{mín}(V_{\text{temp}}, C) - D_t) \quad (10)$$

6.6 Demanda diaria

La demanda diaria D_t se calcula según el tipo de uso y los meses de operación definidos por el usuario:

$$D_t = \begin{cases} N \times d & \text{si el mes } t \text{ está en operación y el día es hábil (o incluye fines de semana)} \\ 0 & \text{en caso contrario} \end{cases} \quad (11)$$

donde N es el número de personas y d es la dotación (L/persona/día).

7 Análisis de Escenarios Críticos y Selección Estocástica

App-SCALL no diseña en base a promedios irreales que diluyen el riesgo. El código analiza la serie histórica real (2000–2020) y extrae **tres escenarios físicos** para someter el sistema a estrés:

7.1 Año Seco (Percentil P5)

Representa el peor escenario pluviométrico del período. El año seco real es aquel cuya precipitación anual corregida más se aproxima al percentil 5 de la distribución histórica:

$$\text{Año seco} = \arg \min_{a \in \mathcal{A}} |PP_a - P_{05}| \quad (12)$$

donde $\mathcal{A}$ es el conjunto de años con al menos 300 días de registro válido.

7.2 Año Lluvioso (Percentil P95)

Análogamente, el año lluvioso es el que más se aproxima al percentil 95:

$$\text{Año lluvioso} = \arg \min_{a \in \mathcal{A}} |PP_a - P_{95}| \quad (13)$$

7.3 Año Normal: Algoritmo de Doble Filtro Estadístico

Para encontrar el año más representativo del comportamiento climático típico, App-SCALL implementa un algoritmo en dos fases que evita seleccionar un año con volumen normal pero distribución temporal atípica (ej. toda la lluvia en un solo mes):

Fase 1 — Filtro Volumétrico: se preseleccionan los 3 años cuya precipitación anual total PP_a es más cercana a la mediana $\tilde{P}P$ del período:

$$\mathcal{C} = 3 \text{ años con menor } |PP_a - \tilde{P}P|, \quad a \in \mathcal{A} \quad (14)$$

Fase 2 — Filtro de Distribución Temporal: para cada candidato $c \in \mathcal{C}$, se calcula la Suma de Errores al Cuadrado (SSE) entre su distribución mensual real y el promedio mensual histórico del período completo:

$$SSE_c = \sum_{i=1}^{12} (Mes_{c,i} - \overline{Mes_i})^2 \quad (15)$$

El sistema selecciona como Año Normal aquel que minimiza el SSE:

$$\text{Año normal} = \arg \min_{c \in \mathcal{C}} SSE_c \quad (16)$$

El uso del SSE en lugar del RMSE tradicional responde a un criterio de eficiencia computacional: garantiza matemáticamente la selección del mismo año óptimo en una fracción del tiempo.

8 Curva de Optimización: El “Sweet Spot” del Diseño

Una de las funciones más avanzadas del software es la **Optimización Automática de Capacidad**. La app genera un vector de capacidades de prueba (desde 1.000 L hasta $\max(30,000 \text{ L}, 2 \times C_{\text{usuario}})$, en pasos de 1.000 L) y ejecuta el algoritmo YAS vectorizado sobre el año simulado para cada capacidad.

8.1 Vectorización del bucle YAS

A diferencia de la simulación principal (que opera día a día con Python puro para preservar el estado del estanque), la curva de optimización simula *todas las capacidades en paralelo* usando operaciones matriciales de NumPy:

Listing 1: Núcleo vectorizado de la curva de optimización

```
caps = np.array(capacidades_prueba, dtype=np.float64) # shape:
      (N,)
nivel = np.zeros(len(caps))

for captado_i, demanda_i in zip(cap_arr, dem_arr):
    disp = np.minimum(nivel + captado_i, caps)
    usada = np.minimum(disp, demanda_i)
    nivel = np.maximum(0.0, disp - demanda_i)
    sum_tot += usada
```

8.2 Identificación del punto óptimo

Una vez calculada la curva de eficiencia $\eta(C)$ (porcentaje de cobertura vs. capacidad), el punto óptimo se determina mediante dos criterios en cascada:

1. **Criterio del 95 %:** si la curva alcanza el 95 % de cobertura, el óptimo es el primer estanque que lo logra:

$$C^* = \min\{C \in \mathcal{C} : \eta(C) \geq 95\%\} \quad (17)$$

2. **Algoritmo Kneedle** (para curvas que no alcanzan el 95 %): se normalizan ambos ejes al intervalo $[0, 1]$ y se busca el punto donde la curva cóncava se desvía más por encima de la diagonal — la zona de máxima ganancia marginal relativa antes del aplanamiento:

$$C^* = \arg \max_{C \in \mathcal{C}} \left(\hat{\eta}(C) - \hat{C} \right) \quad (18)$$

donde $\hat{\cdot}$ denota normalización al rango $[0, 1]$.

9 Mapa de Factibilidad Nacional

La pestaña “Mapa de Factibilidad Nacional” provee contexto espacial para cualquier proyecto en Chile. A diferencia de enfoques que muestran únicamente puntos discretos en las estaciones, App-SCALL genera una **cobertura continua** del territorio mediante triangulación de Delaunay.

9.1 Precipitación media anual por estación

Previo a la triangulación, se calcula la precipitación media anual de cada estación CR² usando solo años con al menos 300 días de registro válido:

$$\overline{PP}_e = \frac{1}{|\mathcal{Y}_e|} \sum_{a \in \mathcal{Y}_e} PP_{e,a} \quad (19)$$

donde $\mathcal{Y}_e$ es el conjunto de años completos de la estación e .

9.2 Triangulación de Delaunay

Sobre las coordenadas (lon_e, lat_e) de las ~ 400 estaciones con datos válidos se construye una triangulación de Delaunay usando `scipy.spatial.Delaunay`. Esta triangulación tiene la propiedad de maximizar el ángulo mínimo de cada triángulo, produciendo una malla espacialmente regular y visualmente coherente.

Para evitar triángulos que crucen zonas sin datos (océano, desierto, frontera), se aplica un filtro de arista máxima:

$$\text{Triángulo válido} \iff \max(d_{12}, d_{23}, d_{13}) \leq 2,2 \quad (20)$$

donde d_{ij} es la distancia euclidiana en grados entre los vértices i y j del triángulo. El umbral de 2,2 equivale aproximadamente a ~ 245 km, suficiente para cubrir Chile central con buena densidad de estaciones y excluir triángulos espurios sobre el océano.

9.3 Valor de cada triángulo

A cada triángulo se le asigna el promedio de la precipitación media anual de sus tres vértices:

$$PP_{\Delta} = \frac{\overline{PP}_{v_1} + \overline{PP}_{v_2} + \overline{PP}_{v_3}}{3} \quad (21)$$

9.4 Escala de colores

El mapa utiliza una escala cromática basada en la paleta de Fundación Amulén:

Color	PP media anual	Zona típica
Blanco casi puro	< 100 mm	Norte Grande (Tarapacá, Antofagasta)
Azul muy claro	100–300 mm	Norte Chico (Atacama, Coquimbo)
Celeste (#76c2f5)	300–800 mm	Zona Central (Valparaíso, Maule)
Azul base (#2e68b1)	800–1.500 mm	La Araucanía, Los Ríos
Azul oscuro (#151434)	> 1.500 mm	Los Lagos, Aysén, Magallanes

9.5 Mapa de cobertura estimada

Un segundo sub-mapa aplica los parámetros de diseño del usuario (superficie de techo, eficiencia y demanda anual) para estimar la cobertura teórica en cada triángulo:

$$\eta_{\Delta} = \min\left(100\%, \frac{PP_{\Delta} \times A \times C_e}{D_{\text{anual}}} \times 100\right) \quad (22)$$

Esta estimación es simplificada (no incluye el efecto del estanque ni la estacionalidad de la demanda) y se presenta como referencia de orden de magnitud.

10 Interfaz de Usuario e Interoperabilidad

La aplicación cuenta con un panel de control interactivo (sidebar) donde el usuario define todos los parámetros de entrada, organizados en secciones:

Sección	Parámetros
Identificación	Nombre de usuario (para historial multi-usuario)
1. Datos del Proyecto	Nombre del proyecto, superficie de techo (m ²), capacidad del estanque (L), eficiencia de captación (%)
Consumo	Número de personas, litros/persona/día, tipo de uso (escuela/residencia), meses de operación, inclusión de fines de semana
2. Calidad de Datos	Umbral mínimo de registros válidos (%)
3. Coordenadas	Latitud/longitud manual o link de Google Maps; altitud manual o automática (API)
Gradiente Orográfico	Porcentaje de variación de lluvia por 100 m de desnivel

10.1 Pestañas de resultados

Los resultados se organizan en cinco pestañas:

1. **Simulación Histórica y Escenarios:** serie continua 2000–2020 con Plotly interactivo, curvas de nivel del estanque para los tres escenarios, curvas de optimización de capacidad, y tablas diarias exportables a Excel.
2. **Resumen Mensual Histórico:** climograma de precipitación mensual promedio de la estación seleccionada.
3. **Mapa de Factibilidad Nacional:** mapa interactivo con triangulación de Delaunay (sub-pestañas: precipitación media y cobertura estimada).
4. **Historial:** listado de simulaciones guardadas del usuario activo, con opciones de recarga, descarga de PDF individual y descarga masiva en ZIP (ver Sección 12).
5. **Dashboard:** panel de estadísticas agregadas de uso de la plataforma (total de simulaciones, usuarios activos, mapa de proyectos).

10.2 Exportación de datos

- **Excel (.xlsx):** balance diario completo (fecha, lluvia, captado, demanda, estanque, rebalse, déficit) para cada escenario climático. Generado con `xlsxwriter`.
- **PDF:** informe de viabilidad con identidad visual Amulén (ver Sección 11).

11 Informe PDF de Viabilidad

App-SCALL genera un informe PDF profesional utilizando la librería `fpdf2`. El documento sigue la identidad visual de Fundación Amulén e incluye:

11.1 Identidad visual

- **Tipografía:** Poppins (Regular, Bold, Italic, BoldItalic) — tipografía corporativa de Amulén, incluida como archivo TTF en el repositorio y registrada dinámicamente en `fpdf2`.
- **Paleta de colores:**
 - Azul Oscuro #151434 — encabezado, pie de página, texto principal
 - Azul Base #2e68b1 — barras de sección, bordes
 - Azul Cielo #76c2f5 — franjas decorativas, acentos
 - Arena #f8f3ea — fondo de filas alternas, texto sobre oscuro

- **Logo:** logo blanco de Amulén embebido en encabezado (upscaled a 1.200 px con Lanczos para evitar pixelación) y en pie de página como marca de agua (600 px).

11.2 Estructura del informe

1. **Encabezado:** fondo azul oscuro con logo, título “SIMULADOR SCALL”, subtítulo y fecha de generación.
2. **Parámetros del sistema:** tabla de 4 columnas con todos los inputs del usuario.
3. **Estación meteorológica:** nombre, código, altitud, distancia efectiva y calidad de datos.
4. **Resultados por escenario:** tabla comparativa (año seco / normal / lluvioso) con lluvia total, volumen captado, demanda, cobertura (%) y días sin agua.
5. **Gráfico de nivel del estanque:** serie diaria del año normal generada con Matplotlib, embebida como PNG.
6. **Gráfico de curva óptima del estanque:** curva cobertura vs. capacidad generada con Matplotlib, que incluye marcadores del estanque actual (línea naranja) y el punto óptimo recomendado (marcador verde), embebida como PNG inmediatamente después de la tabla de KPIs.
7. **Tamaño óptimo recomendado:** tabla de KPIs con estanque ingresado, tamaño óptimo, cobertura actual y cobertura óptima.
8. **Conclusiones y veredicto:** texto automático con clasificación de viabilidad:
 - **VIABLE** si cobertura en año normal $\geq 60\%$
 - **PARCIALMENTE VIABLE** si cobertura $\geq 30\%$
 - **NO RECOMENDADO** si cobertura $< 30\%$
9. **Pie de página automático:** aparece en todas las páginas con texto institucional y logo Amulén.

12 Historial de Simulaciones y Persistencia de Datos

A partir de la versión 2.1, App-SCALL implementa un sistema de persistencia de simulaciones en la nube mediante **Supabase** (PostgreSQL gestionado), lo que permite a múltiples usuarios acceder, recuperar y comparar simulaciones pasadas desde cualquier dispositivo.

12.1 Arquitectura de persistencia

El módulo `history.py` gestiona toda la comunicación con Supabase usando la **API REST** directamente mediante `requests`, evitando la dependencia del cliente oficial

`supabase-py` (incompatible con Python 3.14 por requerir compilación de extensiones C++). Las credenciales se leen en orden de prioridad:

1. `.streamlit/secrets.toml` → sección `[supabase]` con `url` y `key`.
2. Variables de entorno `SUPABASE_URL` / `SUPABASE_KEY` (útil para despliegues en CI/CD).

El archivo `secrets.toml` está excluido del repositorio mediante `.gitignore`; en producción las credenciales se gestionan desde la interfaz de Streamlit Community Cloud.

12.2 Esquema de la base de datos

La tabla `simulaciones` en Supabase almacena tanto metadatos escalares (para consultas rápidas) como el informe completo serializado en formato `jsonb`:

Columna	Tipo	Descripción
<code>id</code>	<code>bigint</code>	Clave primaria autoincremental
<code>fecha_simulacion</code>	<code>timestampz</code>	Timestamp UTC automático
<code>usuario</code>	<code>text</code>	Nombre ingresado por el usuario
<code>nombre_proyecto</code>	<code>text</code>	Identificador del proyecto
<code>pct_cubierto_normal</code>	<code>float8</code>	Cobertura en año normal (%)
<code>dias_sin_agua_normal</code>	<code>integer</code>	Días de déficit en año normal
<code>cap_optima</code>	<code>float8</code>	Capacidad óptima recomendada (L)
<code>informe_datos_json</code>	<code>jsonb</code>	Informe completo: DataFrames diarios, curvas, metadatos

12.3 Serialización de datos

Los objetos Python no serializables directamente a JSON se convierten mediante dos mecanismos:

- **DataFrames Pandas:** `df.to_json(orient='records', date_format='iso')` produce una cadena JSON embebida en el campo `jsonb`.
- **Tipos NumPy** (`numpy.float64`, `numpy.int64`): se convierten a tipos Python nativos mediante una función `_json_default(obj)` pasada como argumento `default` a `json.dumps()`.
- **Tuplas de curvas:** la tupla (`capacidades`, `eficiencias`, `cap_opt`, `ef_act`, `ef_opt`) se serializa como diccionario con listas Python nativas.

12.4 Gestión multi-usuario

Cada usuario ingresa su nombre en el sidebar antes de simular. Este nombre se almacena en el campo `usuario` de cada registro y se usa como filtro en la consulta de historial (`?usuario=eq.{nombre}`), garantizando que cada persona vea únicamente sus propias simulaciones.

Si no se ingresa nombre, la simulación se guarda bajo el alias “Anónimo”. Para evitar re-guardados espurios ante cada interacción con la interfaz, se emplea la bandera de sesión `_sim_guardada` que se activa tras el primer guardado exitoso y se resetea solo cuando el usuario presiona el botón de cálculo.

12.5 Funcionalidades del historial

- **Autoguardado:** cada simulación ejecutada se persiste automáticamente sin acción del usuario.
- **Recarga:** al cargar una simulación previa, el sistema reconstituye el estado completo de la sesión (`informe_datos`, `resultado`) y muestra un banner informativo en Tab 1.
- **PDF individual:** generación y descarga del informe PDF de cualquier simulación del historial.
- **Descarga ZIP masiva:** botón “Descargar todo (ZIP)” que genera todos los PDFs del usuario y los comprime en un archivo `SCALL_{usuario}_informes.zip`, con cada archivo nombrado según el proyecto correspondiente.
- **Eliminación:** con confirmación de dos pasos para evitar borrados accidentales.

13 Análisis de Seguridad y Autonomía Hídrica

El software implementa una capa de interpretación de resultados basada en umbrales de seguridad operativa, clasificando la disponibilidad del recurso en tres zonas:

Zona	Nivel del estanque	Significado operativo
Zona de Alerta	$< 10\%$	Riesgo de “reserva muerta” y succión de sedimentos
Zona Operativa	$10\% - 80\%$	Rango de funcionamiento normal del sistema
Zona de Seguridad	$> 80\%$	Resiliencia y autonomía total frente a períodos sin llu

Adicionalmente, para cada escenario climático el sistema reporta:

- **Días sin agua:** número de días en que el estanque llega a cero y el consumo no puede satisfacerse.
- **Porcentaje de cobertura:** fracción del consumo total anual que es cubierta por el sistema de captación.
- **Volumen de rebalse total:** agua captada que no puede almacenarse por límite de capacidad del estanque (útil para dimensionar el sistema de evacuación).

14 Arquitectura del Software

Archivo	Rol
<code>app_scall.py</code>	Aplicación principal Streamlit: UI, visualizaciones Plotly, generación de PDF e informe Excel, tabs Historial y Dashboard
<code>simulator.py</code>	Motor de simulación: balance diario YAS, selección de años extremos (P5/P95/SSE), curva de optimización Kneedle
<code>data_loader.py</code>	Carga y filtrado de datos CR ² , control de calidad por estación, caché Streamlit
<code>utils.py</code>	Helpers: formato chileno de números, distancia Haversine vectorizada, corrección de coordenadas
<code>history.py</code>	Historial de simulaciones: persistencia en Supabase vía REST API, serialización/deserialización JSON, gestión multi-usuario
<code>requirements.txt</code>	Dependencias del proyecto
<code>.streamlit/secrets.toml</code>	Credenciales Supabase (excluido del repositorio vía <code>.gitignore</code>)
<code>.env.example</code>	Plantilla de variables de entorno para despliegues alternativos
<code>fonts/</code>	Tipografía Poppins (Regular, Bold, Italic, BoldItalic) en formato TTF
<code>data/cr2_prDaily_2020.txt.gz</code>	Dataset principal: precipitación diaria por estación (5,3 MB comprimido)

14.1 Stack tecnológico

Librería	Uso principal
<code>streamlit</code>	Framework de aplicación web interactiva
<code>pandas</code>	Manipulación de datos tabulares
<code>numpy</code>	Cálculo vectorizado (simulación, Haversine, Kneedle)
<code>plotly</code>	Visualizaciones interactivas (mapas, series de tiempo, curvas)
<code>scipy</code>	Triangulación de Delaunay (<code>scipy.spatial.Delaunay</code>)
<code>matplotlib</code>	Gráfico de nivel del estanque para el PDF
<code>fpdf2</code>	Generación de informes PDF
<code>Pillow</code>	Upscaling del logo (Lanczos) para alta resolución en PDF
<code>xlsxwriter</code>	Exportación de balances diarios a Excel
<code>requests</code>	Consultas a APIs externas (altitud, coordenadas) y Supabase REST API
<code>Supabase</code>	Base de datos PostgreSQL en la nube para historial de simulaciones

15 Despliegue

La aplicación está desplegada en **Streamlit Community Cloud** (plan gratuito):

- **URL:** <https://scallclauda-a6ziabnwkurju5e24brqke.streamlit.app>
- **Repositorio:** delsholz/SCALL_claude en GitHub (rama main)
- **Actualización:** el despliegue se actualiza automáticamente con cada `git push` a la rama principal.
- **Disponibilidad:** accesible desde cualquier dispositivo (PC, tablet, móvil) mediante navegador web.
- **Limitación:** la app hiberna tras un período de inactividad y tarda ~ 30 segundos en despertar la primera vez. La RAM disponible es 1 GB.

16 Conclusión

App-SCALL se posiciona en la frontera entre la ciencia de datos climáticos y la ingeniería hidrológica aplicada, superando las limitaciones de las calculadoras estáticas tradicionales. Al abandonar los promedios mensuales genéricos en favor de simulaciones empíricas diarias basadas en 20 años de registros reales — incluyendo la Mega-Sequía histórica de Chile Central — la plataforma somete los diseños a pruebas de estrés reales, entregando un nivel de certidumbre sin precedentes para el dimensionamiento a escala local.

La integración de una arquitectura de software robusta que abarca desde el saneamiento automatizado de la base de datos CR², pasando por el modelamiento físico YAS con corrección orográfica, la selección estocástica de escenarios mediante el algoritmo SSE, la optimización de capacidad con el algoritmo Kneedle, y la visualización territorial continua mediante triangulación de Delaunay, asegura que la capacidad óptima sugerida responda fielmente a la física del entorno.

Al traducir millones de registros meteorológicos en indicadores operativos claros de Seguridad y Autonomía Hídrica, App-SCALL democratiza el acceso a la ingeniería de precisión, permitiendo a proyectistas, arquitectos y usuarios finales diseñar sistemas de captación de aguas lluvias que no solo son técnicamente sólidos, sino también económicamente eficientes y climáticamente resilientes frente al cambiante escenario hídrico de Chile.

Referencias Bibliográficas

Referencias

- [1] Fewkes, A. (1999). *The use of predictive models for the sizing of rainwater harvesting systems for use in buildings*. Building and Environment, 34(5), 545–552.
- [2] Jenkins, D., Pearson, F., Moore, E., Sun, J.K., & Valentine, R. (1978). *Feasibility of rainwater collection systems in California*. California Water Resources Center, University of California.
- [3] Centro de Ciencia del Clima y la Resiliencia (CR)². (2020). *Base de datos consolidada de precipitación diaria (cr2_prDaily_2020)*. Universidad de Chile. Disponible en: <http://www.cr2.cl/datos-de-precipitacion/>
- [4] Satopaa, V., Albrecht, J., Irwin, D., & Raghavan, B. (2011). *Finding a “Kneedle” in a Haystack: Detecting Knee Points in System Behavior*. 31st International Conference on Distributed Computing Systems Workshops (ICDCSW).
- [5] Delaunay, B. (1934). *Sur la sphère vide*. Bulletin de l’Académie des Sciences de l’URSS, Classe des Sciences Mathématiques et Naturelles, 6, 793–800.